

Task 6: The lobby of an online game

Tasks 6 and 7 use a new tool, the **condition variable**. Read this introduction first.

Condition variables: how they work

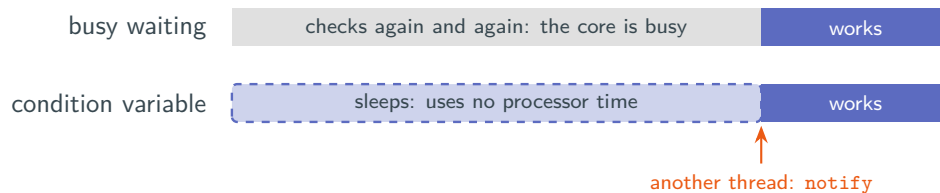
Often a thread has to wait until something happens: until all players are here, until a frame is ready, until it is its turn. With what you know so far, the thread checks the condition in a loop:

```
m.lock();
while (!ready) {           // not yet? look again
    m.unlock();           // let the others change ready
    m.lock();
}
m.unlock();
```

This works, but the thread does nothing useful and still uses a whole core of the processor: it asks again and again, millions of times per second. This is called **busy waiting**.

Condition variable

A **condition variable** (`std::condition_variable`) lets a thread **sleep** until another thread wakes it up. A sleeping thread uses no processor time.



A condition variable always comes with three things:

1. The **state** that the threads wait for (for example a counter of players), protected by a **mutex**.
2. The **waiting** thread. It locks the mutex and checks the condition in a `while` loop. If the condition is false, it calls `wait()`. `wait()` unlocks the mutex and puts the thread to sleep, in one step. When the thread is woken up, `wait()` locks the mutex again and returns.
3. The **waking** thread. It locks the mutex, changes the state, and calls `notify_one()` (wakes up one waiting thread) or `notify_all()` (wakes up all of them).

```

std::mutex m;
std::condition_variable cv;
bool ready = false;

void waiter() {
    std::unique_lock<std::mutex> lock(m);
    while (!ready)           // while, not if: check again after waking up
        cv.wait(lock);      // unlock m and sleep; lock m again when woken up
    // here ready == true and m is locked
}

void waker() {
    std::lock_guard<std::mutex> lock(m);
    ready = true;
    cv.notify_all();         // wake up everybody who waits on cv
}

```

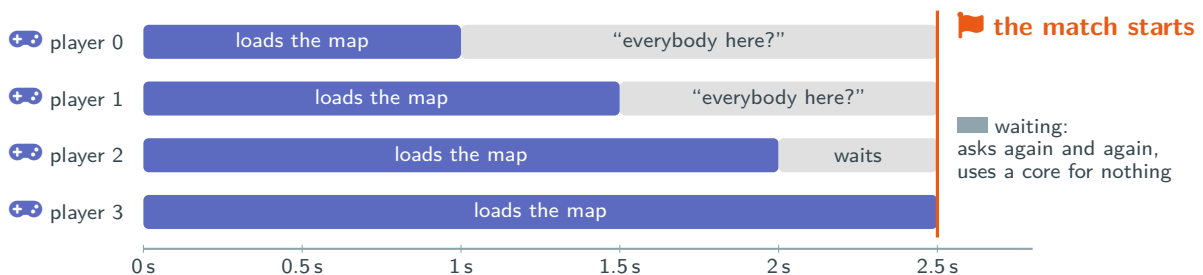
Three details matter:

- **unique_lock, not lock_guard.** `wait()` has to unlock and lock the mutex again. A `lock_guard` cannot do that; a `unique_lock` can.
- **while, not if.** After waking up, the condition may be false again: another thread may have been faster, and a thread can even wake up without any notify (a *spurious wakeup*). So always check again.
- **Check the condition under the mutex.** If the waiter checked without the mutex, the waker could change the state and call `notify` between the check and `wait()`. Nobody would be waiting yet, the notify would be lost, and the waiter would sleep for ever (a *lost wakeup*).

A mutex protects the data. A condition variable lets a thread sleep until the data changes.

The task

A match starts when all 4 players have loaded the map. Every player is a thread that loads the map (it takes a different time for every player) and then waits for the others. Now the players ask again and again: “is everybody here?”, and the game server uses its whole processor just for waiting.



Ready code (in `lobby.h`, we do not look inside)

Functions

load_map(id)	the player loads the map: player 0 takes 1s, player 1 takes 1.5s, ..., player 3 takes 2.5s
start_match(id)	the match starts for this player (prints a line)
Constants	
PLAYERS	4 players in a match

The template is in [Task6](#). This is lobby.cpp:

```
int loaded = 0;           // how many players have loaded the map
std::mutex m;

void player(int id) {
    load_map(id);
    m.lock();
    loaded++;
    while (loaded < PLAYERS) {    // not everybody? ask again
        m.unlock();
        m.lock();
    }
    m.unlock();
    start_match(id);
}

int main() {
    std::vector<std::thread> threads;
    for (int id = 0; id < PLAYERS; id++)
        threads.emplace_back(player, id);
    for (auto &t : threads)
        t.join();
}
```

In the task folder, make run compiles the program and runs it.

Your tasks

- 1 Run it with `time ./build/lobby` (Linux, macOS). How much processor time (user and sys) did the players use, when they mostly waited?
- 2 Replace busy waiting with a `std::condition_variable`. Who should wake the players up? Run it with `time` again.
- 3 `notify_one()` or `notify_all()`? Why?
- 4 A match has 3 rounds, and before each round we wait until everybody is ready. Is it enough to set loaded back to 0 when a round starts? Write it and run it. What happens, and why? How can you fix it?

`time`

real is the time on the clock. user is the processor time spent in your program, and sys the processor time spent in the operating system for your program. On Windows, compare the processor usage in the Task Manager instead.